

PARSER

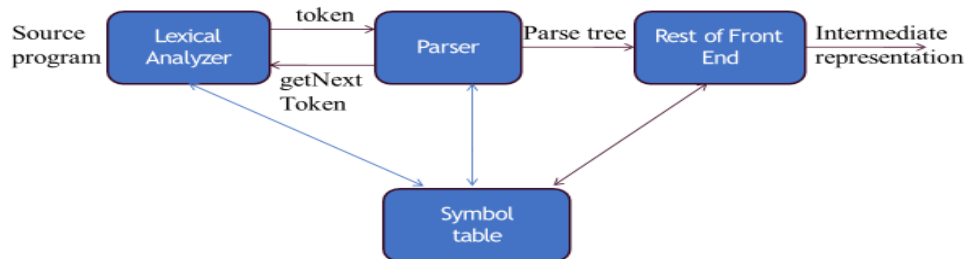
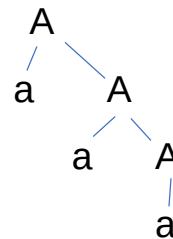
The process of deriving the string from a given grammar using the given productions of the grammar is known as PARSING.

Parser is that phase of compiler which takes token string as input and with the help of existing grammar, converts it into the corresponding parse tree. Parser is also known as Syntax Analyzer.

If we get a success in generating the syntax tree, then it means there is no grammatical mistakes in the string otherwise it means a grammatical error.

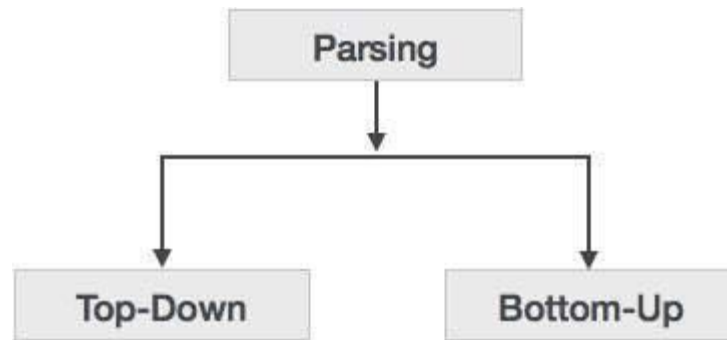
$A \rightarrow Aa/a$

String aaa



Types of Parsing

Syntax analyzers follow production rules defined by means of context-free grammar. The way the production rules are implemented (derivation) divides parsing into two types: top-down parsing and bottom-up parsing.



(A)Top-down Parser:

A Top-down parser tries to create a parse tree from the root towards the leaves scanning

Top-down parser is the parser which generates parse for the given input string with the help of grammar productions by expanding the non-terminals i.e. it starts from the start symbol and ends on the terminals.

It can be also viewed as finding a leftmost derivation (LMD) for an input string.

Example 1

Production rules:

$E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow id$

Input string: id + id * id

The left-most derivation is:

$E \rightarrow E * E$

$E \rightarrow E + E * E$

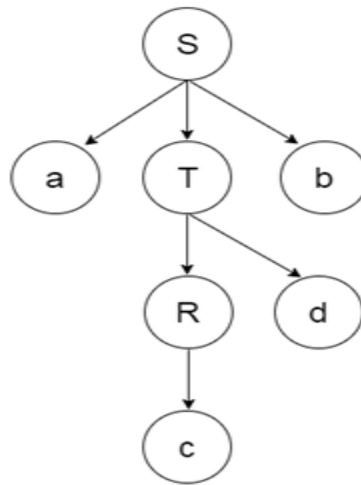
$E \rightarrow id + E * E$

$E \rightarrow id + id * E$

$E \rightarrow id + id * id$

Example 2

Parse Tree representation of input string "acdb" is as follows:



The top down parsing is known as recursive parsing or predictive parsing.

In the top down parsing, the parsing starts from the start symbol and transform it into the input symbol.

Recursive descent parsing: It is a common form of top-down parsing. It is called recursive as it uses recursive procedures to process the input. Recursive descent parsing suffers from backtracking.

Backtracking: It means, if one derivation of a production fails, the syntax analyzer restarts the process using different rules of same production. This technique may process the input string more than once to determine the right production.

(B). Bottom-up Parsing:

Bottom-up Parser is the parser which generates the parse tree for the given input string with the help of grammar productions by compressing the non-terminals i.e. it starts from non-terminals and ends on the start symbol. It uses reverse of the right most derivation.

As the name suggests, bottom-up parsing starts with the input symbols and tries to construct the parse tree up to the start symbol.

Example:

Input string: $a + b * c$

Production rules:

$S \rightarrow E$ $E \rightarrow E + T$
--

$$\begin{aligned} E &\rightarrow E * T \\ E &\rightarrow T \\ T &\rightarrow \text{id} \end{aligned}$$

Let us start bottom-up parsing

$$a + b * c$$

Read the input and check if any production matches with the input:

$$\begin{aligned} a + b * c \\ T + b * c \\ E + b * c \\ E + T * c \\ E * c \\ E * T \\ E \\ S \end{aligned}$$

Production

$$\begin{aligned} E &\rightarrow T \\ T &\rightarrow T * F \\ T &\rightarrow \text{id} \\ F &\rightarrow T \\ F &\rightarrow \text{id} \end{aligned}$$

2. Parse Tree representation of input string "id * id" is as follows:

id * id

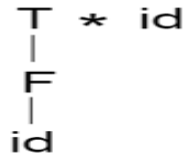
Step 1

$$\begin{array}{c} F * \text{id} \\ | \\ \text{id} \end{array}$$

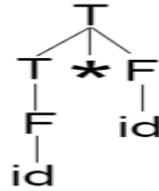
Step 2

$$\begin{array}{c} T * \text{id} \\ | \\ F \\ | \\ \text{id} \end{array}$$

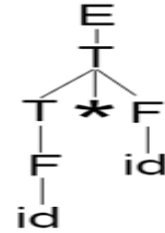
Step 3



Step 4



Step 5



Step 6

Bottom up parsing is also known as shift-reduce parsing.

Bottom up parsing is used to construct a parse tree for an input string.

In the bottom up parsing, the parsing starts with the input symbol and construct the parse tree up to the start symbol by tracing out the rightmost derivations of string in reverse. Bottom-up parser is classified into 2 types: LR parser, and Operator precedence parser.

(i). LR parser:

LR parser is the bottom-up parser which generates the parse tree for the given string by using unambiguous grammar. It follow reverse of right most derivation.

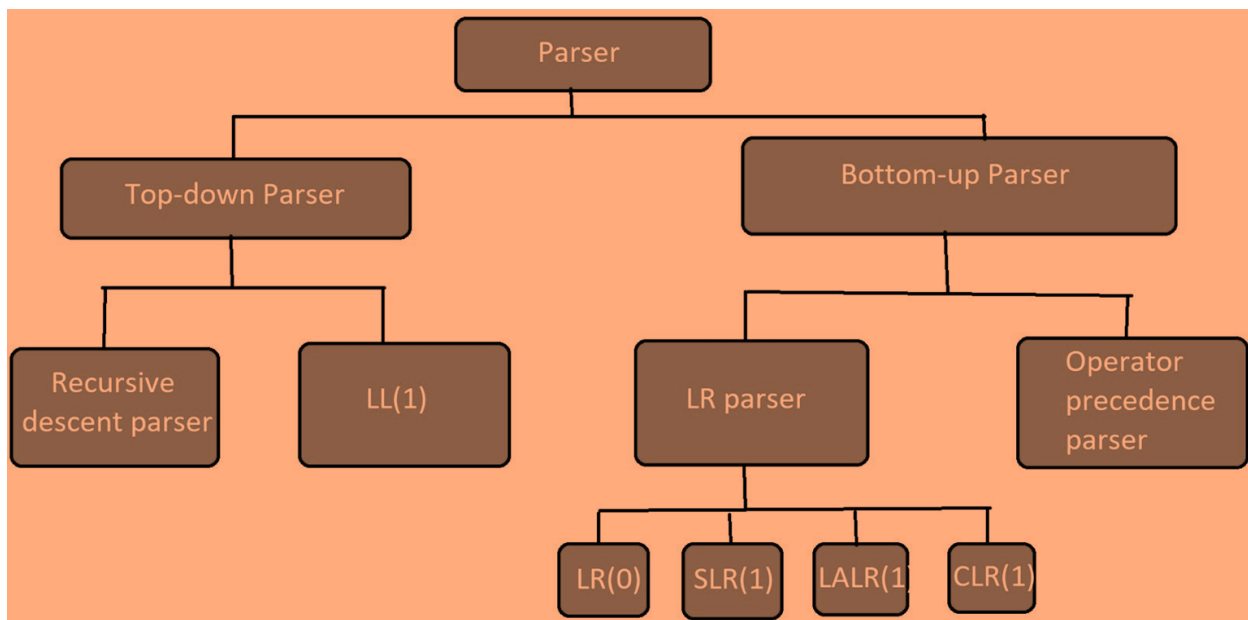
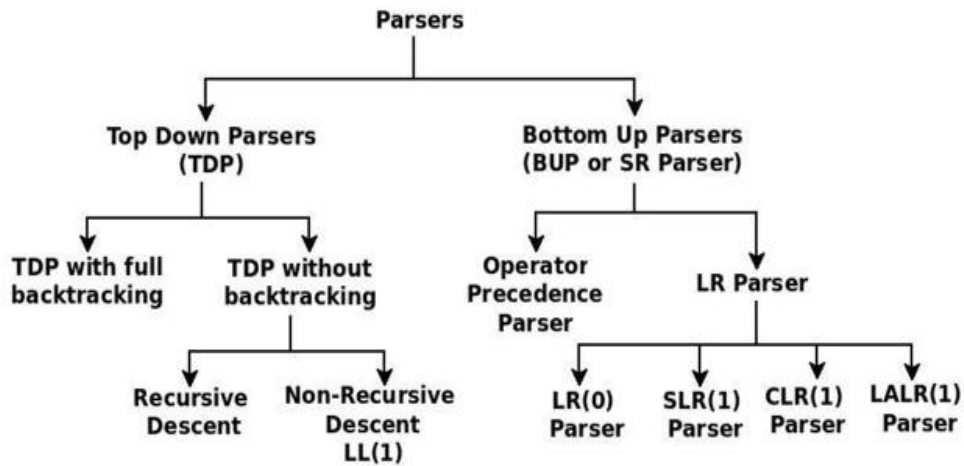
LR parser is of 4 types:

1. LR (0)
2. SLR(1)
3. LALR(1)
4. CLR(1)

(ii). Operator precedence parser:

It generates the parse tree form given grammar and string but the only condition is two consecutive non-terminals and epsilon never appear in the right-hand side of any production.

TYPES OF PARSER



Top Down parser without backtracking is also called predictive parser. It predicts right/ correct production at first time and no chance of backtracking. Two main types are there

Recursive Descent Parser

Non Recursive Descent Parser

(i). Recursive descent parser:

It is a kind of Top-Down Parser. A top-down parser builds the parse tree from the top to down, starting with the start non-terminal. A *Predictive Parser* is a special case of Recursive Descent Parser, where no Back Tracking is required.

It is also known as Brute force parser or the with backtracking parser. It basically generates the parse tree by using brute force and backtracking.

Recursive descent is a top-down parsing technique that constructs the parse tree from the top and the input is read from left to right. It uses procedures for every terminal and non-terminal entity. This parsing technique recursively parses the input to make a parse tree, which may or may not require back-tracking. But the grammar associated with it (if not left factored) cannot avoid back-tracking.

In recursive descent parser we used recursion.

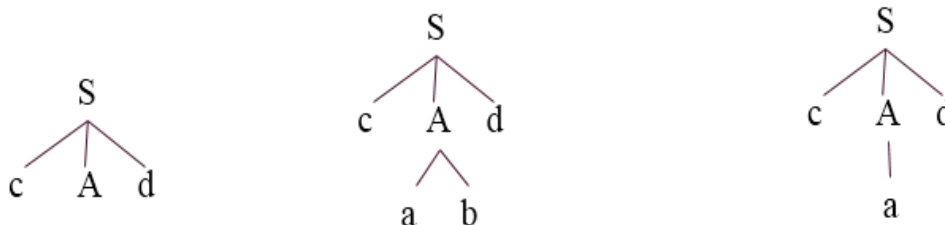
It has the capability to predict which production is to be used to replace the input string. The predictive parser does not suffer from backtracking.

Production rules:

S $\rightarrow$ cAd

A $\rightarrow$ ab | a

Input string: cad



Back-tracking

Top- down parsers start from the root node (start symbol) and match the input string against the production rules to replace them (if matched). To understand this, take the following example of CFG:

S $\rightarrow$ rXd | rZd

X $\rightarrow$ oa | ea

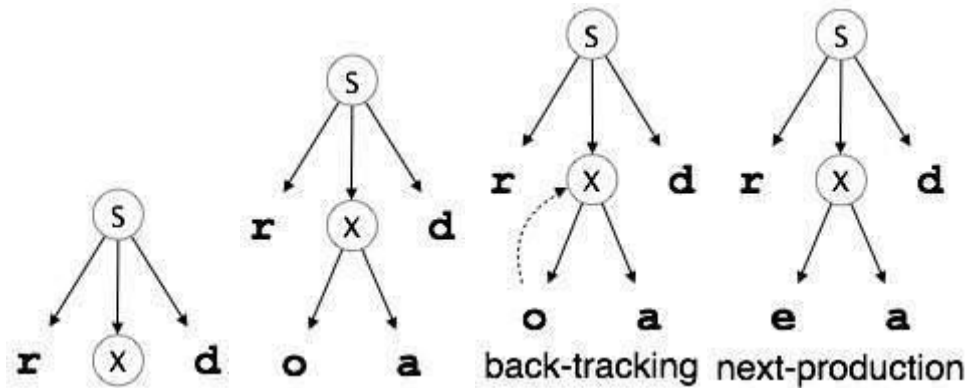
Z $\rightarrow$ ai

For an input string: **read**, a top-down parser, will behave like this:

It will start with S from the production rules and will match its yield to the left-most letter of the input, i.e. 'r'. The very production of S ($S \rightarrow rXd$) matches with it. So the top-down parser advances to the next input letter (i.e. 'e'). The parser tries to expand non-terminal

'X' and checks its production from the left ($X \rightarrow oa$). It does not match with the next input symbol. So the top-down parser backtracks to obtain the next production rule of X, ($X \rightarrow ea$).

Now the parser matches all the input letters in an ordered manner. The string is accepted.



(ii). Non-recursive descent parser:

It is also known as LL(1) parser or predictive parser or without backtracking parser or dynamic parser, which has the capability to predict which production is to be used to replace the input string. It uses parsing table to generate the parse tree instead of backtracking. To accomplish its tasks, the predictive parser uses a look-ahead pointer, which points to the next input symbols.

In this parser we are not using recursion we create parse tree through stack.

We use LL(1) grammar in LL(1) parser.

For LL(1) parser the grammar should not be left recursive or left factorize.

LL(1)

The first L means scanning input from left to right

The second L means leftmost derivation

And 1 stands for using one input symbol for look ahead

LL(1) PARSER

For LL(1) parser we have to make First and Follow of a variable.

From First and Follow we generate parsing table.

Parsing table is used to satisfy any string using stack.

Why FIRST?

If the compiler would have come to know in advance, that what is the “first character of the string produced when a production rule is applied”, and comparing it to the current

character or token in the input string it sees, it can wisely take decision on which production rule to apply.

Let's take the grammar :

$S \rightarrow cAd$

$A \rightarrow bc|a$

And the input string is "cad".

Thus, in the example above, if it knew that after reading character 'c' in the input string and applying $S \rightarrow cAd$, next character in the input string is 'a', then it would have ignored the production rule $A \rightarrow bc$ (because 'b' is the first character of the string produced by this production rule, not 'a'), and directly use the production rule $A \rightarrow a$ (because 'a' is the first character of the string produced by this production rule, and is same as the current character of the input string which is also 'a').

Hence it is validated that if the compiler/parser knows about first character of the string that can be obtained by applying a production rule, then it can wisely apply the correct production rule to get the correct syntax tree for the given input string.

Why FOLLOW?

The parser faces one more problem. Let us consider below grammar to understand this problem.

$A \rightarrow aBb$

$B \rightarrow c | \epsilon$

And suppose the input string is "ab" to parse.

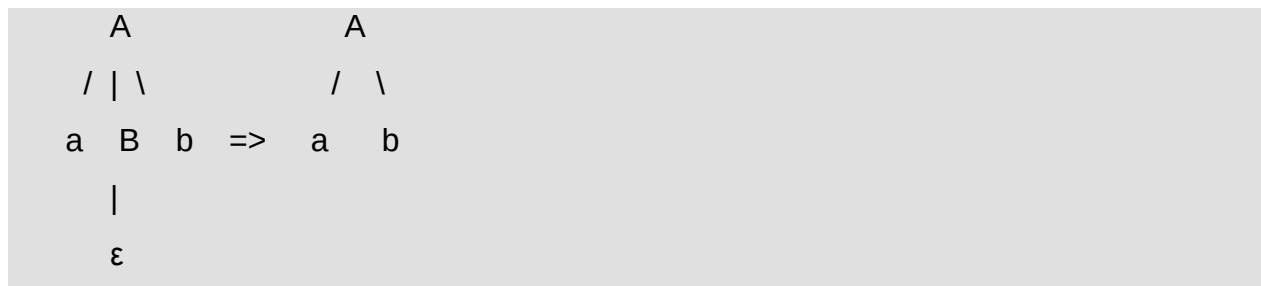
As the first character in the input is a, the parser applies the rule $A \rightarrow aBb$.

```
  A
 / | \
a  B  b
```

Now the parser checks for the second character of the input string which is b, and the Non-Terminal to derive is B, but the parser can't get any string derivable from B that contains b as first character.

But the Grammar does contain a production rule $B \rightarrow \epsilon$, if that is applied then B will vanish, and the parser gets the input "ab", as shown below. But the parser can apply it only when it knows that the character that follows B in the production rule is same as the current character in the input.

In RHS of $A \rightarrow aBb$, b follows Non-Terminal B , i.e. $\text{FOLLOW}(B) = \{b\}$, and the current input character read is also b . Hence the parser applies this rule. And it is able to get the string “ab” from the given grammar.



So FOLLOW can make a Non-terminal to vanish out if needed to generate the string from the parse tree.

The conclusion is, we need to find FIRST and FOLLOW sets for a given grammar, so that the parser can properly apply the needed rule at the correct position.

FIRST Set in Syntax Analysis

$\text{FIRST}(X)$ for a grammar symbol X is the set of terminals that begin the strings derivable from X .

Rules to compute FIRST set:

1. If x is a terminal, then $\text{FIRST}(x) = \{ 'x' \}$
2. If $x \rightarrow \epsilon$, is a production rule, then add ϵ to $\text{FIRST}(x)$.
3. If $X \rightarrow Y_1 Y_2 Y_3 \dots Y_n$ is a production,
 1. $\text{FIRST}(X) = \text{FIRST}(Y_1)$
 2. If $\text{FIRST}(Y_1)$ contains ϵ then $\text{FIRST}(X) = \{ \text{FIRST}(Y_1) - \epsilon \} \cup \{ \text{FIRST}(Y_2) \}$
 3. If $\text{FIRST}(Y_i)$ contains ϵ for all $i = 1$ to n , then add ϵ to $\text{FIRST}(X)$.

FOLLOW Set in Syntax Analysis

- 1) $\text{FOLLOW}(S) = \{ \$ \}$ // where S is the starting Non-Terminal
- 2) If $A \rightarrow pBq$ is a production, where p , B and q are any grammar symbols, then everything in $\text{FIRST}(q)$ except ϵ is in $\text{FOLLOW}(B)$.
- 3) If $A \rightarrow pB$ is a production, then everything in $\text{FOLLOW}(A)$ is in $\text{FOLLOW}(B)$.
- 4) If $A \rightarrow pBq$ is a production and $\text{FIRST}(q)$ contains ϵ , then $\text{FOLLOW}(B)$ contains $\{ \text{FIRST}(q) - \epsilon \} \cup \text{FOLLOW}(A)$